# Module 3

# Machine learning (ML)

**Cynthia Sandor**

Edmond J Safra Assistant Professor
UKRI Future Leader Fellow
UK Dementia Research Institute Group Leader

UK Dementia Research Institute
Imperial College London
THE MICHAEL J. FOX FOUNDATION FOR PARKINSON'S RESEARCH
Founding funders:
UKRI Medical Research Council
EDMOND J. SAFRA PHILANTHROPIC FOUNDATION

# What We'll Do Today

- **Lecture:** Introduction to machine learning
- **Tutorial 4 / Challenge, with Marirena Bafaloukou:** build and test machine-learning models on plasma proteomics data

Task 1: predict tremor severity (regression). Task 2: predict Parkinson's disease (classification). Task 3: find structure without labels (PCA, UMAP and k-means).

Challenge: estimate the age of the intestine from plasma proteins.

# Lecture Plan

## 1 What is machine learning?

- Where machine learning sits in AI
- From statistics to machine learning
- Learning rules from data
- Supervised and unsupervised
- The data and the pipeline

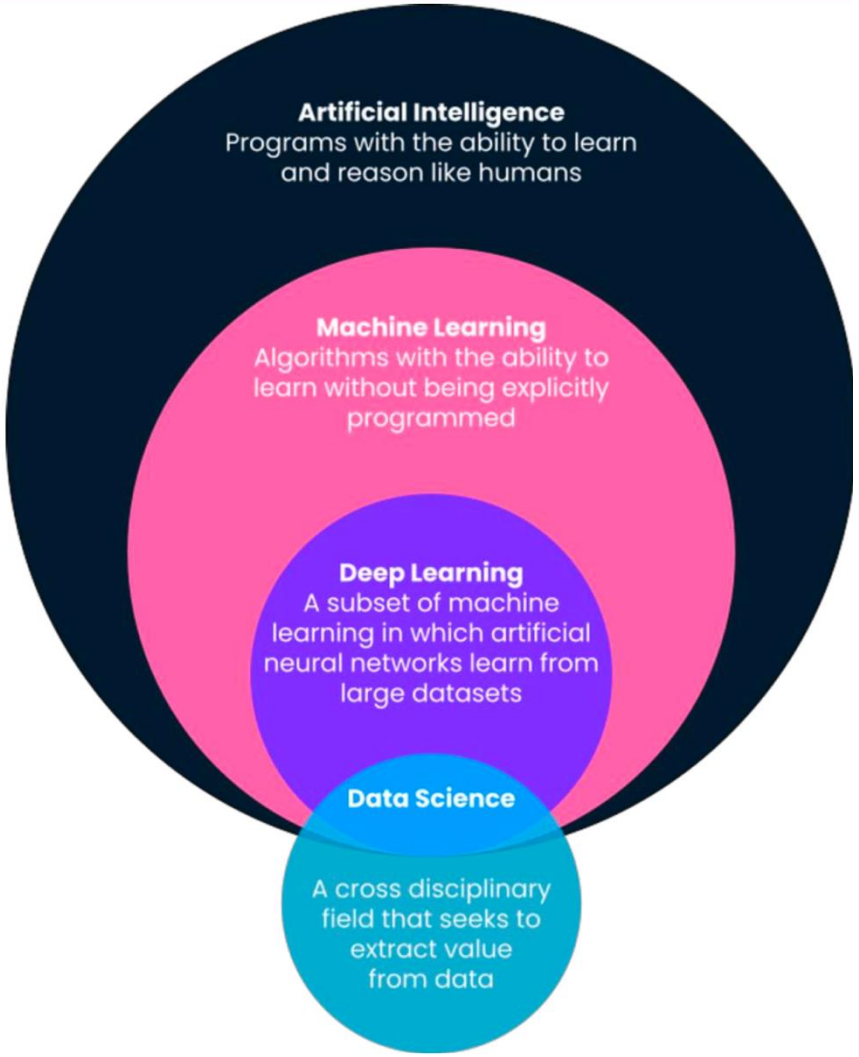
## 2 Supervised learning

- Train–test split
- Missing values and scaling
- Regression: tremor severity
- Classification: Parkinson's or control
- Feature selection
- Confusion matrix and metrics

## 3 Unsupervised learning

- Dimensionality reduction
- PCA and UMAP
- How many components?
- k-means clustering
- How many clusters?

# Where Does Machine Learning Sit?



# From Statistics to Machine Learning

$$y = \gamma + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3$$

outcome = intercept + coefficients × features

## Statistics: explain

- We know the outcome, e.g. LEDD, and estimate the coefficients
- Which features are significantly associated with the outcome?
- How strong is the relationship? (p-values)

## Machine learning: predict

- The model learns the best coefficients from the data
- Then uses them to predict the outcome for new patients
- Judged by how well it predicts unseen data

**Same equation, different goal: inference versus prediction.**

# Traditional Coding vs Machine Learning

## Traditional coding: you write the rule

Pass if a student studies 3 hours or more

```
def will_pass(hours):
    if hours >= 3:
        return "Pass"
    else:
        return "Fail"

print(will_pass(5))   # Pass
print(will_pass(1))   # Fail
```

## Machine learning: the rule is learnt

Show examples; the computer finds the rule

```
from sklearn.linear_model import LogisticRegression
import numpy as np

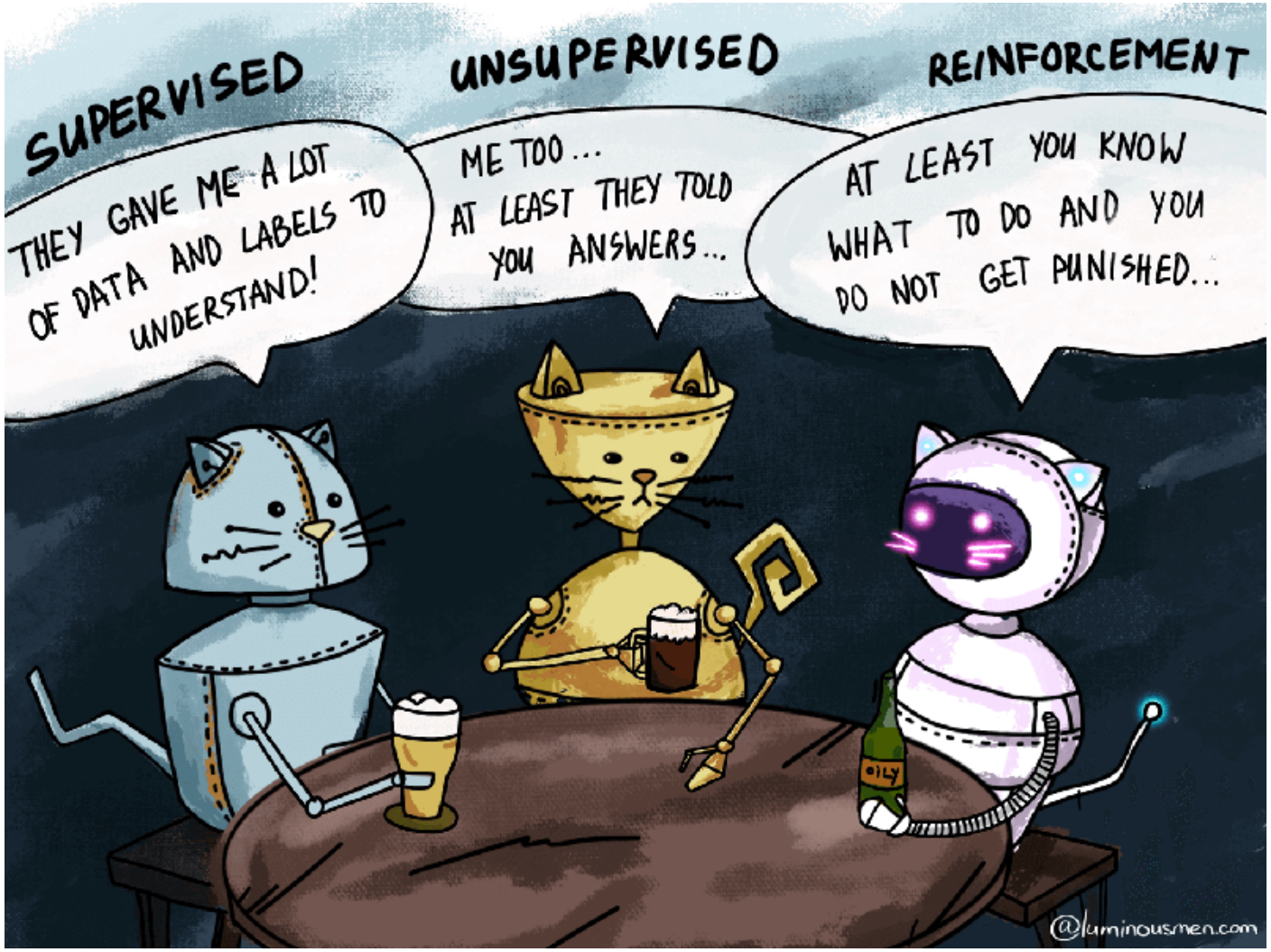
# Training data: hours studied
X = np.array([[1], [2], [3], [4], [5]])
y = np.array([0, 0, 0, 1, 1])  # 1 = Pass, 0 = Fail

model = LogisticRegression()
model.fit(X, y)

# Predict for a new student
print(model.predict([[2.5]]))  # -> [0] (Fail)
print(model.predict([[4.5]]))  # -> [1] (Pass)
```

**In traditional coding, I program the rule. In machine learning, I show examples and the computer discovers the rule.**

# Types of Machine Learning



# Supervised and Unsupervised Learning

## Supervised: learns from labels

- Each example has input features and a known label
- The model learns f(features) → label
- **Regression:** predict a number, e.g. tremor severity
- **Classification:** predict a class, e.g. Parkinson's or control
- Trained on one set, tested on unseen data

## Unsupervised: no labels

- Finds structure in the data on its own
- **Dimensionality reduction:** PCA, UMAP
- **Clustering:** k-means

# The Data: Plasma Proteomics in Parkinson's

## Tasks 1 and 2

- 500 people, about 30% with Parkinson's
- 40 plasma proteins (Olink, NPX values)
- Age and sex
- Tremor severity from wearable sensors, 0 to 10
- One sample per person

## Task 3

- 500 people, about 40% with Parkinson's
- 209 proteins from the Olink Neurology panel; a few are below detection in everyone
- Age and sex
- Tremor severity, 0 to 4 (all controls score 0)

**Synthetic data, modelled on real Parkinson's cohorts.**

# The Machine Learning Pipeline

**1 Prepare**

Numeric types; encode categories (one-hot)

**2 Split**

Training and test sets

**3 Missing values**

Remove or impute

**4 Scale**

Mean 0, SD 1

**5 Select features**

Keep informative ones (Task 2)

**6 Train**

Fit the model

**7 Interpret**

Read the coefficients

**8 Evaluate**

Predict the unseen test set

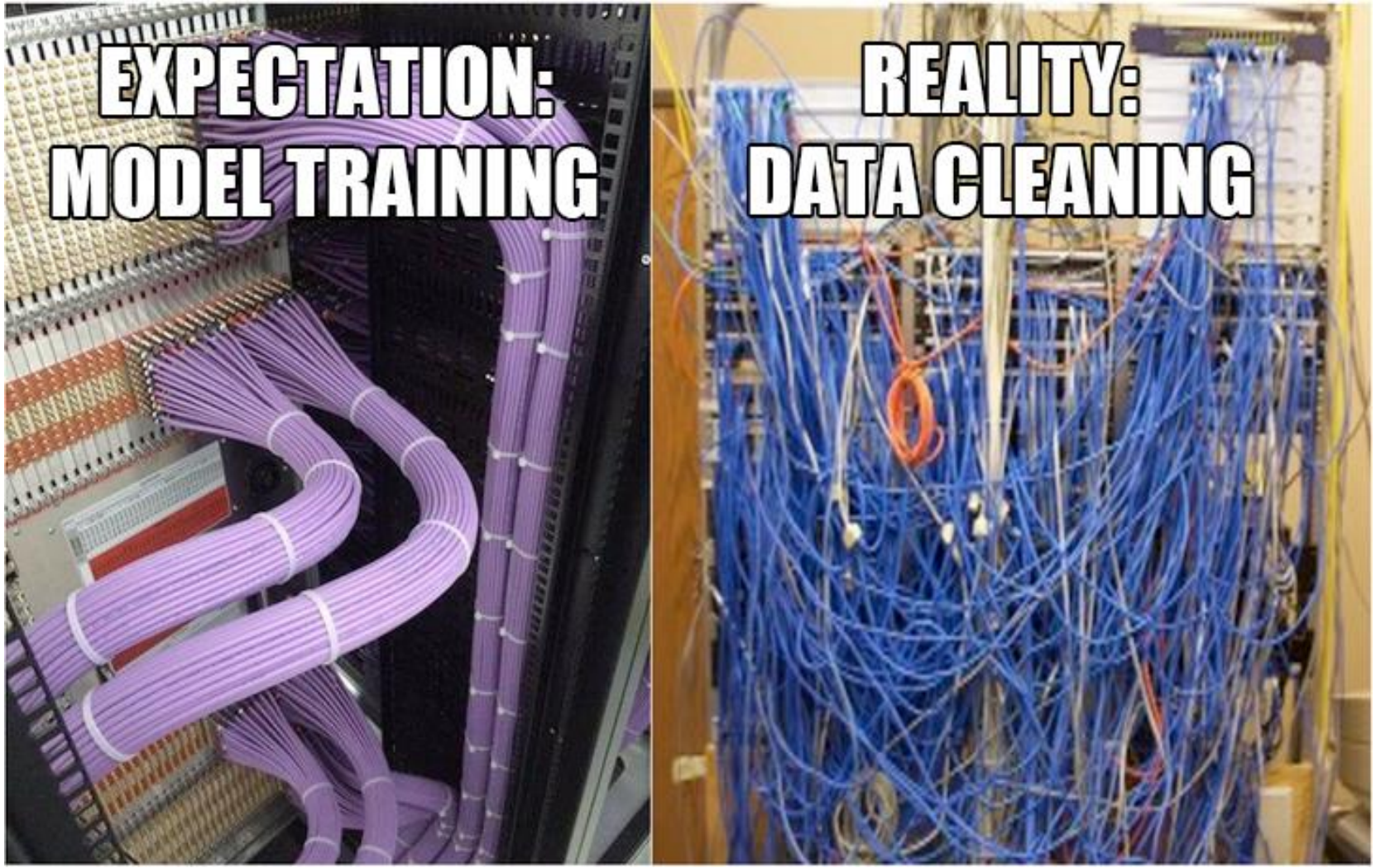
**Steps 3 to 5 (gold) are fitted on the training set only, then applied to the test set.**

# Step 1: Split into Training and Test Sets




- **Training set:** the model learns the patterns
- **Test set:** held out and never seen during training
- The test set shows how well the model generalises to new patients
- Usually 70/30 or 80/20; set random_state so the split can be reproduced

# Garbage In → Garbage Out



# Missing Values and Scaling

## Missing values

- Most scikit-learn models cannot use missing values
- Remove features or samples with too many missing values
- Impute the rest: mean, median, mode, or model-based (Tutorial 3)

## Scaling

- Features with larger values can dominate the model
- StandardScaler: each feature to mean 0 and SD 1

## Fit on the training set only, then apply to the test set

`scaler.fit_transform(X_train)` then `scaler.transform(X_test)`

Otherwise information from the test set leaks into the model and inflates its performance.

# What Happens If We Skip Data Cleaning?




### Q1 – Duplicates
You trained a Parkinson's model and got 98% accuracy, but half the rows were duplicated.
👉 What happens when you test on new patients?

### Q2 – Missing values
Some protein values are missing for several participants.
👉 What happens if you feed NaNs directly into the model?

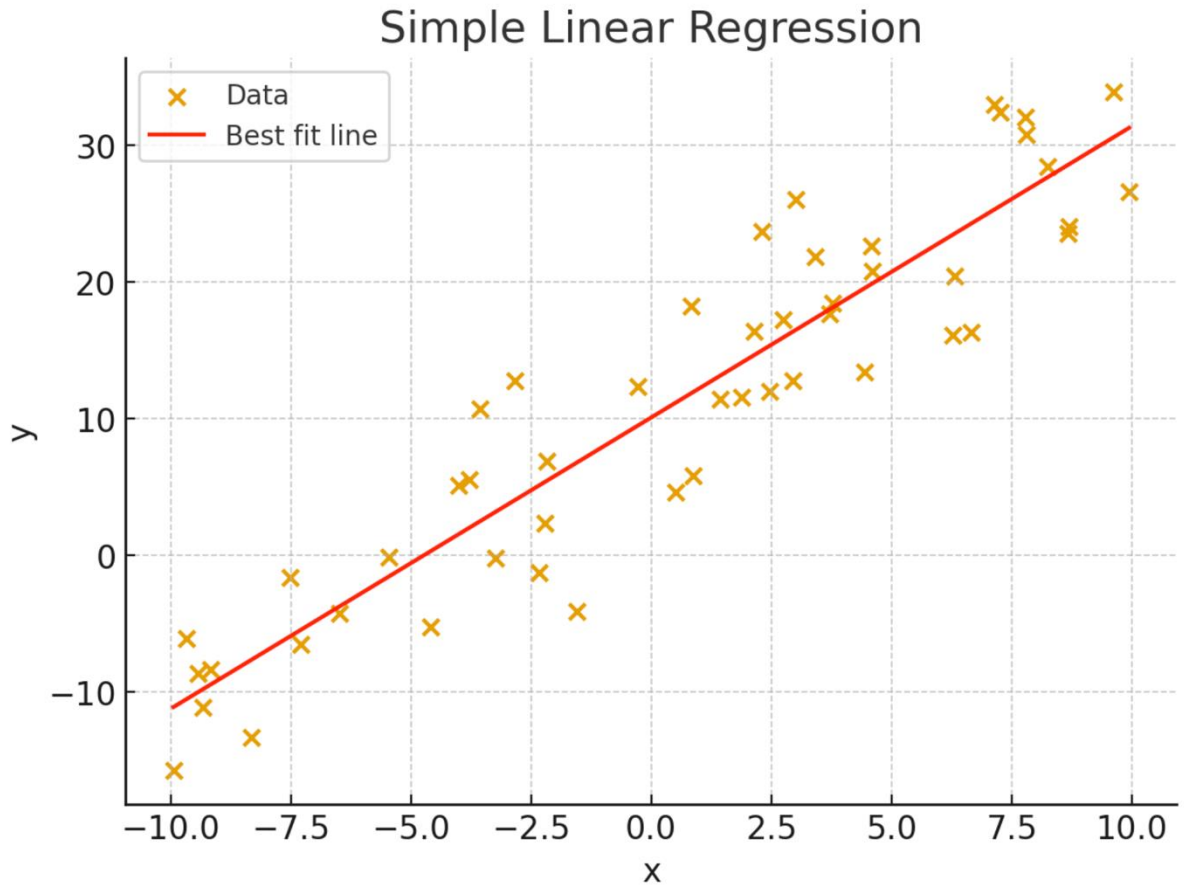
### Q3 – Unscaled features
Age ranges from 40 to 85, while a protein's NPX values vary by only a few units.
👉 Without scaling, which feature dominates?

### Q4 – Outliers
One participant's protein value is 1,000 times higher than anyone else's: clearly an error.
👉 How could this affect your model?

### Q5 – Inconsistent formatting
The age column mixes numbers and the text "unknown".
👉 What happens during preprocessing?

# Supervised ML: Regression

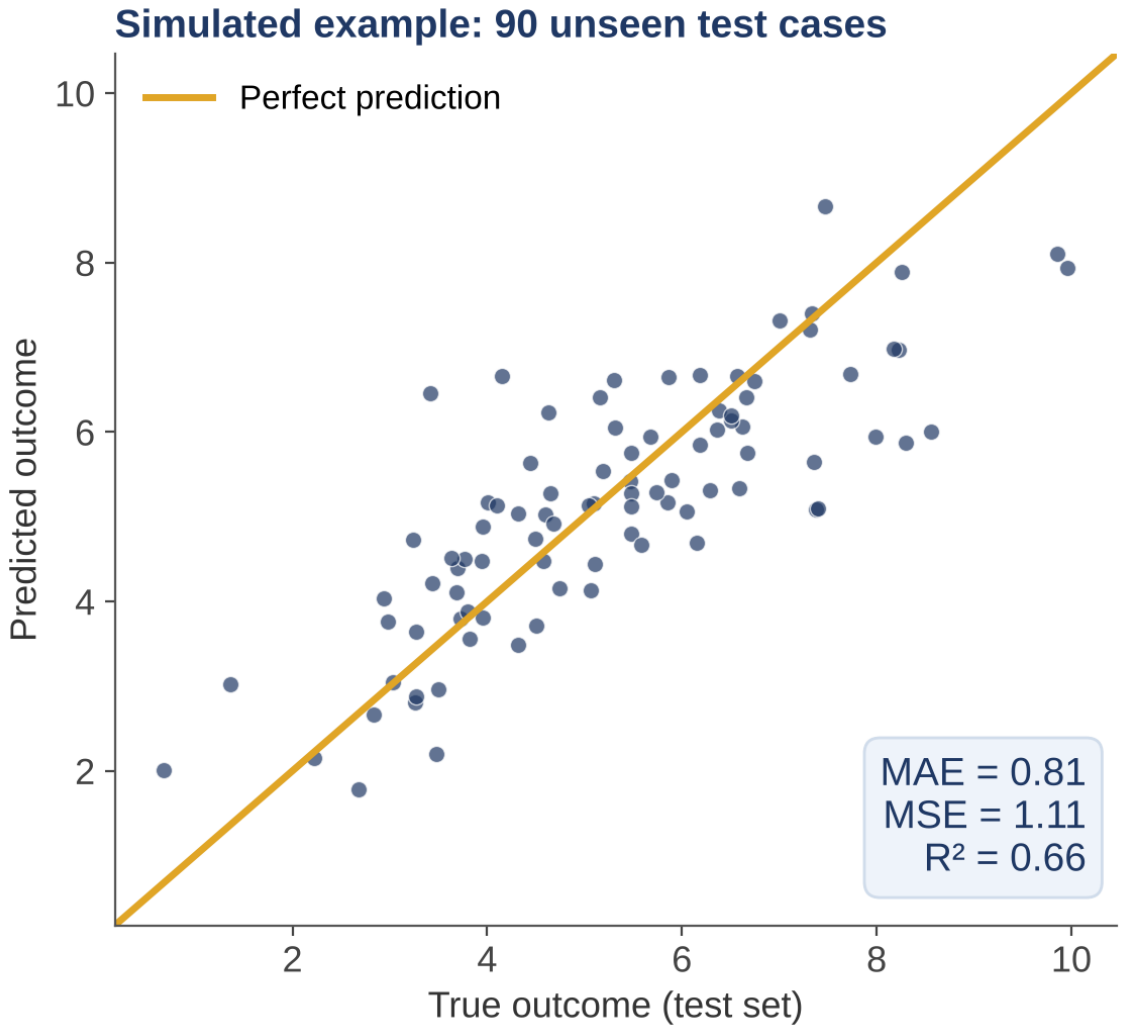
The **simplest and most widely known machine learning method** is **Linear Regression**.




Predict **motor severity (UPDRS score)** using **age**:

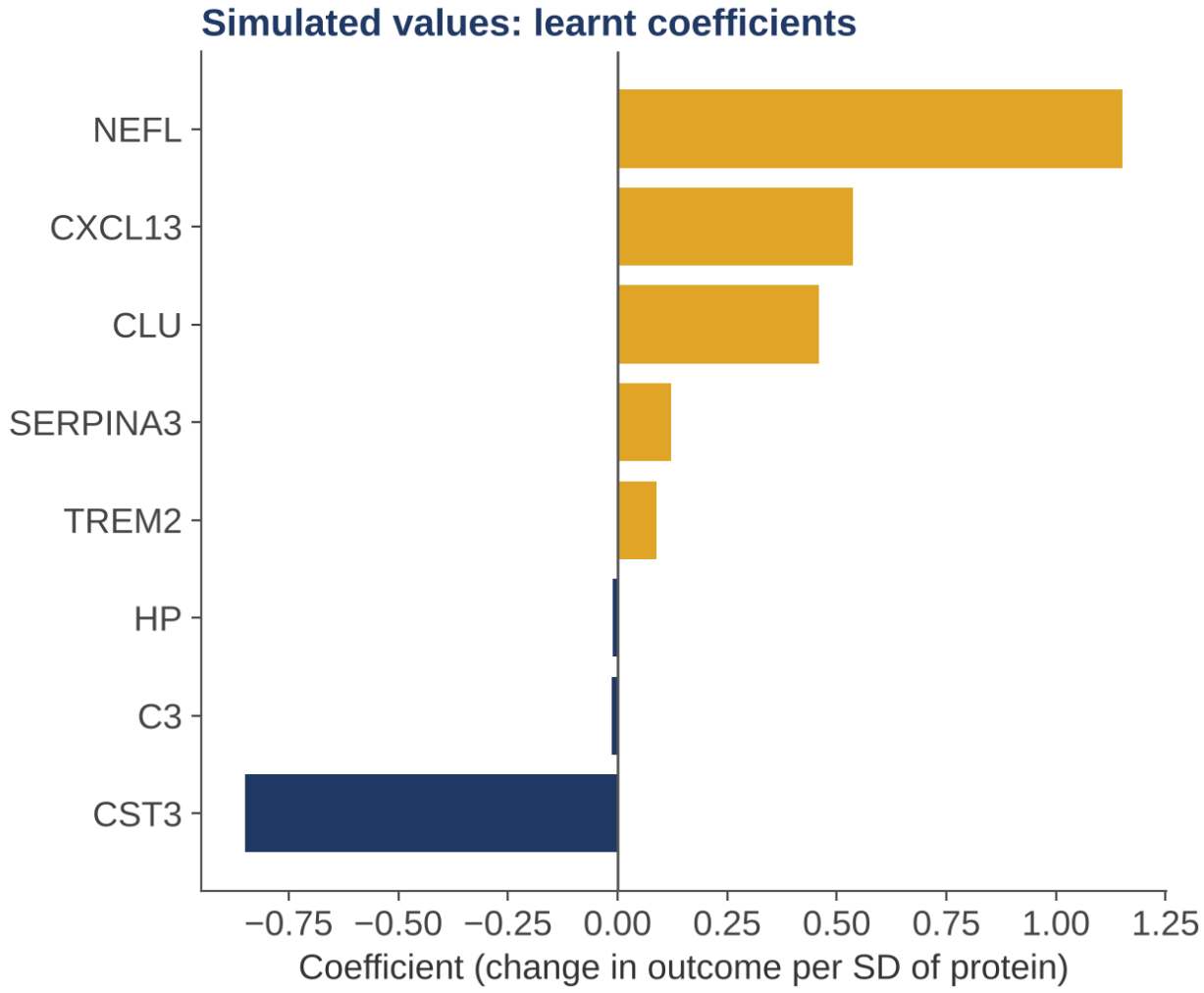
**y = 10 + 2.13x**

# Evaluating a Regression Model




- **MAE:** the average size of the errors, in the outcome's units
- **MSE:** the average squared error, so large errors count more
- **$R^2$:** the proportion of the variation in the outcome that the model explains
- Points close to the line are good predictions

# Interpreting the Model: Coefficients



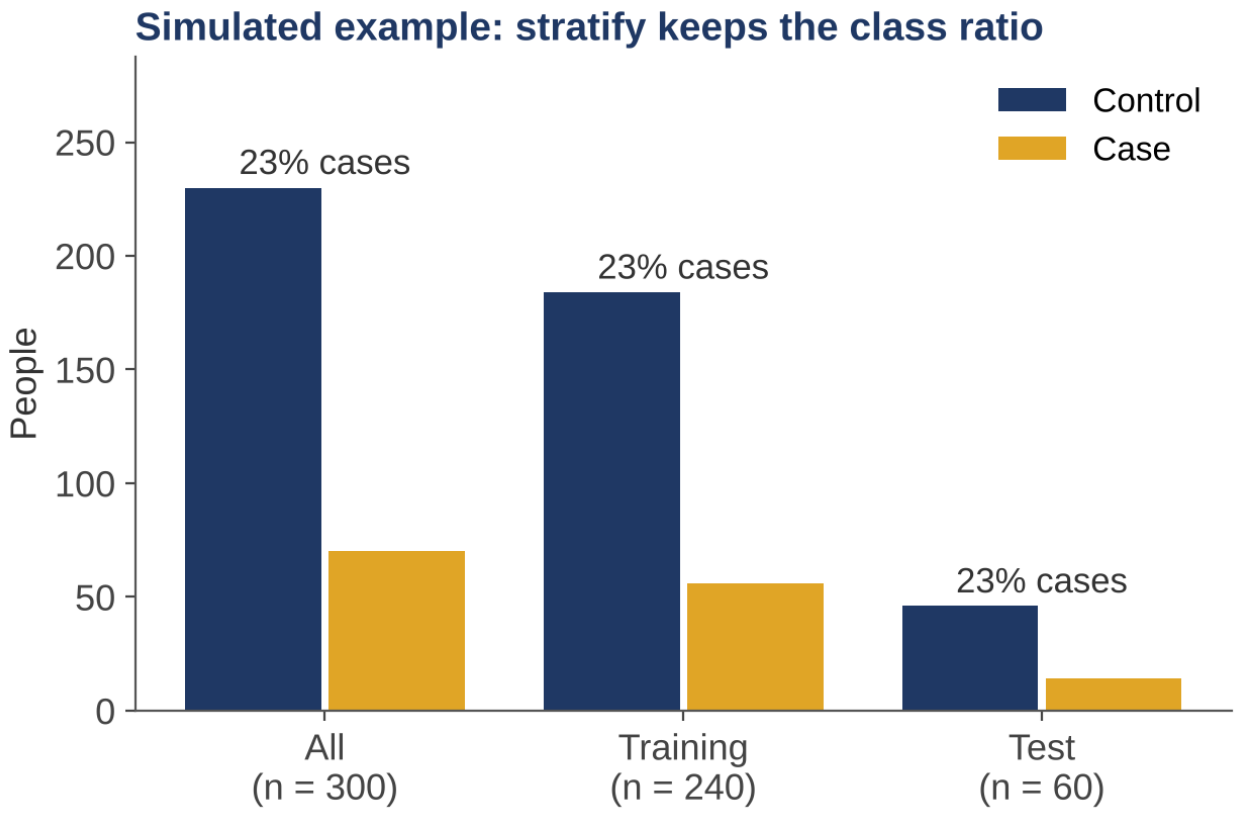

- Each coefficient is the change in the predicted outcome for a 1 SD increase in that feature
- **Positive:** higher values, higher prediction
- **Negative:** higher values, lower prediction
- **Close to 0:** little effect once the other features are in the model

# Regression vs Classification

| Regression | Classification |
| --- | --- |
| Predicts continuous values | Predicts discrete classes |
| Task 1: tremor severity | Task 2: disease status (0 = control, 1 = Parkinson's) |
| Linear regression | Logistic regression |
| Evaluated with MAE, MSE, $R^2$ | Evaluated with the confusion matrix |

**The model decides the task: LogisticRegression treats 0 and 1 as two classes, not as numbers on a scale.**

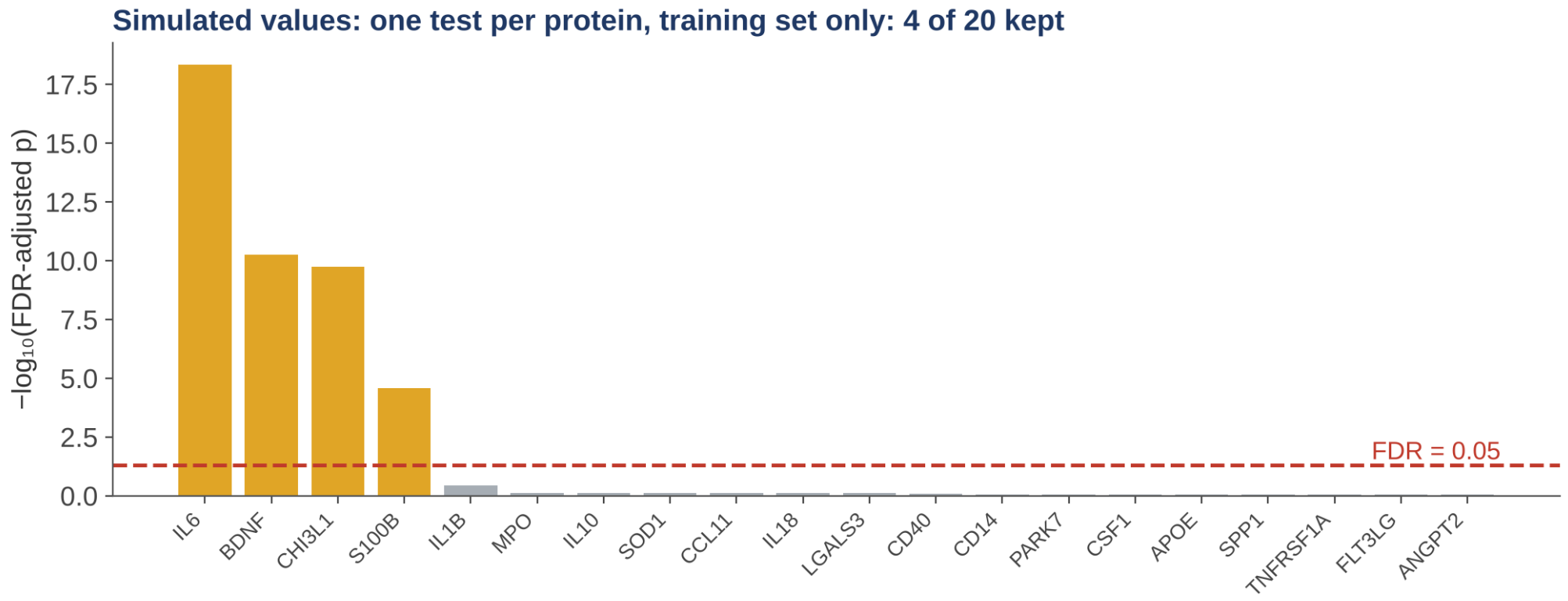
# Task 2: Know Your Data Before You Split



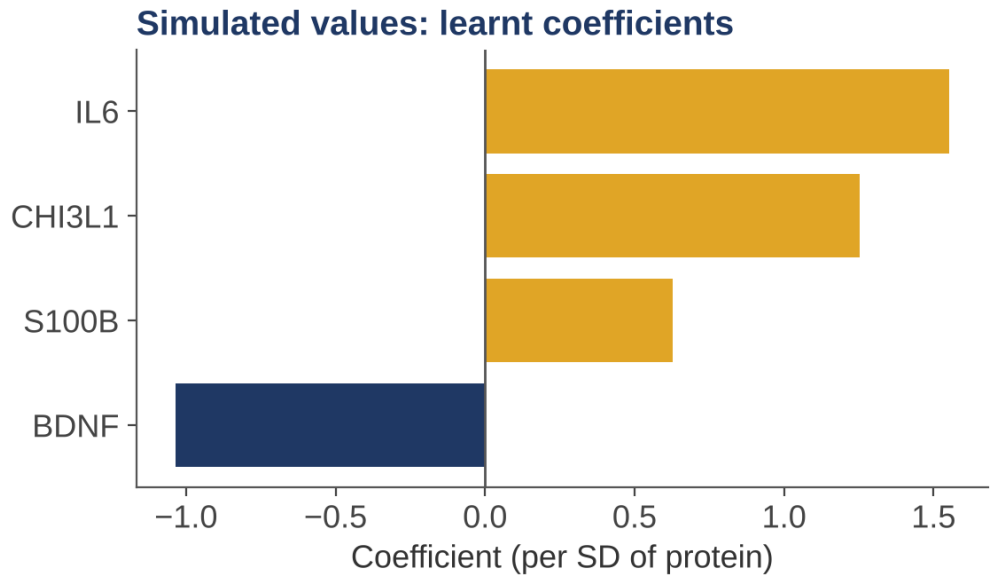

- **How many samples per person?** Here, one.
- Several samples from one patient are not independent: keep them all in the same set, or the test set contains patients the model has already seen
- **How balanced are the classes?** Count them before you split
- **stratify=y** keeps the same ratio in the training and test sets

# Feature Selection: Keep the Informative Proteins

- Uninformative features add noise and can harm performance through overfitting
- Filter methods test each feature against the outcome, on the training set only

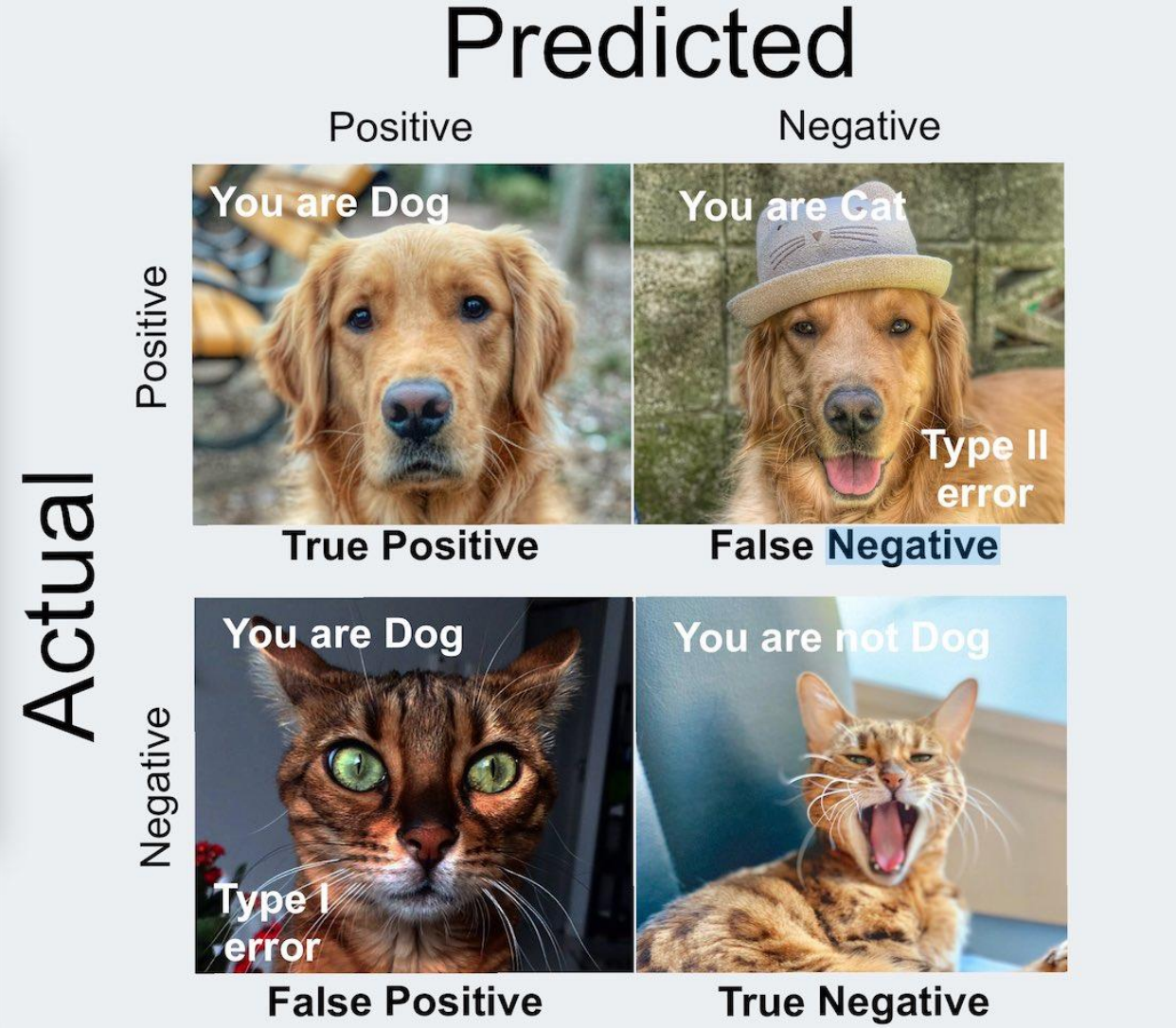


# Training a Classifier



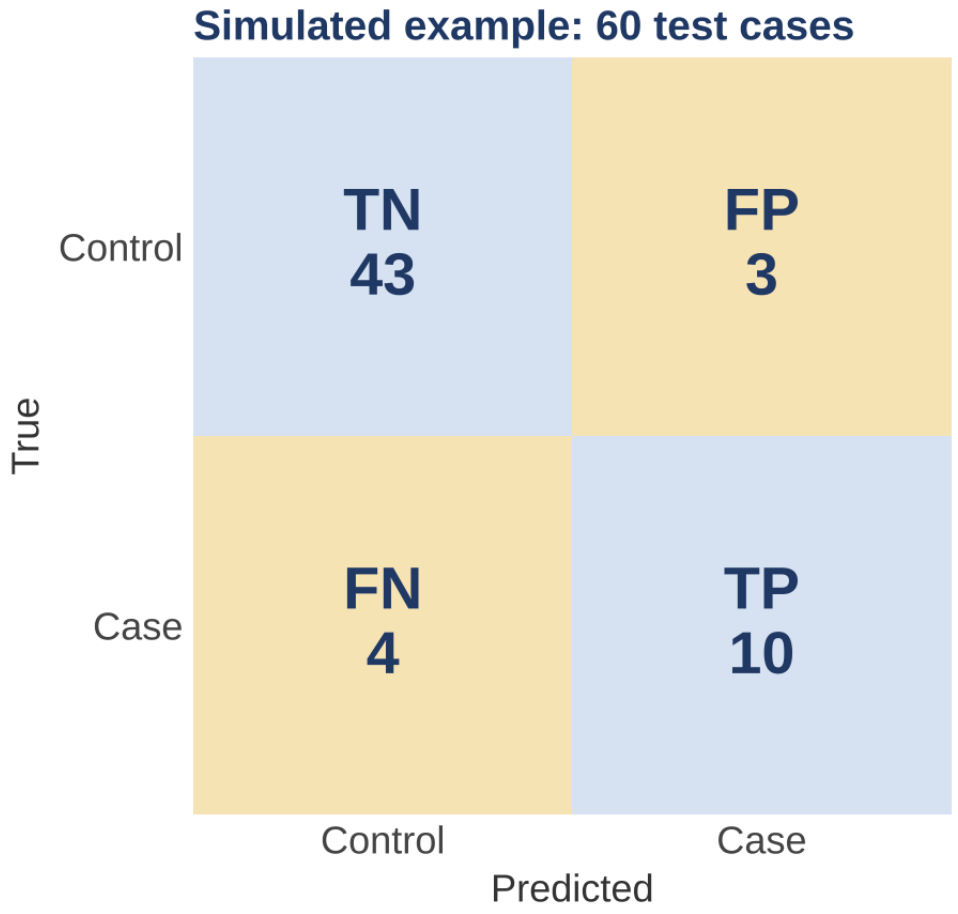

- **Logistic regression:** a classifier that learns one coefficient per feature
- Trained on the scaled training set, it predicts 0 or 1 for each test case
- **Positive coefficient:** higher values push the prediction towards the case class
- **Negative coefficient:** higher values push it towards control

# Confusion matrix




A **confusion matrix** is a table used to measure how well a classification model performs

# Summary Metrics




| Metric | Formula | Example |
|---|---|---|
| **Accuracy** | (TP + TN) / all | 53/60 = 0.88 |
| **Sensitivity (recall)** | TP / (TP + FN) | 10/14 = 0.71 |
| **Specificity** | TN / (TN + FP) | 43/46 = 0.93 |
| **Precision** | TP / (TP + FP) | 10/13 = 0.77 |

**Which error matters more in diagnosis: a false positive or a false negative?**

# Common Pitfalls

## Data leakage

- Scale, impute and select features on the training set only
- Keep each patient's samples in the same set

## Class imbalance

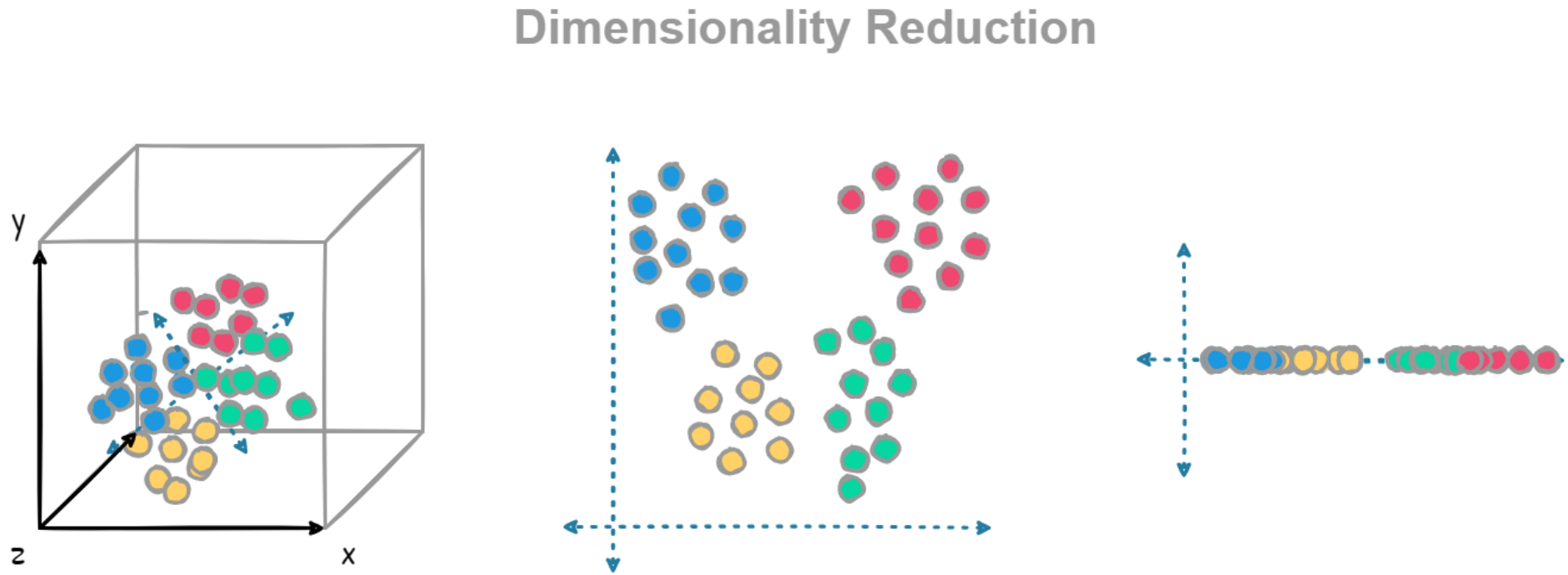
- Check the class counts before splitting
- Use stratify to keep the same ratio in both sets

## Too many features

- Uninformative features add noise and can cause overfitting
- Select or reduce the features: the curse of dimensionality

# Unsupervised ML: Dimensionality Reduction

Transforms data into a **lower-dimensional space** to simplify analysis and visualization.



# Two Ways to Reduce Dimensions

## Feature selection

- Keep a subset of the original features
- Example: the 3 proteins that differ between Parkinson's and controls (Task 2)
- Risk: dropping features can lose biological signal

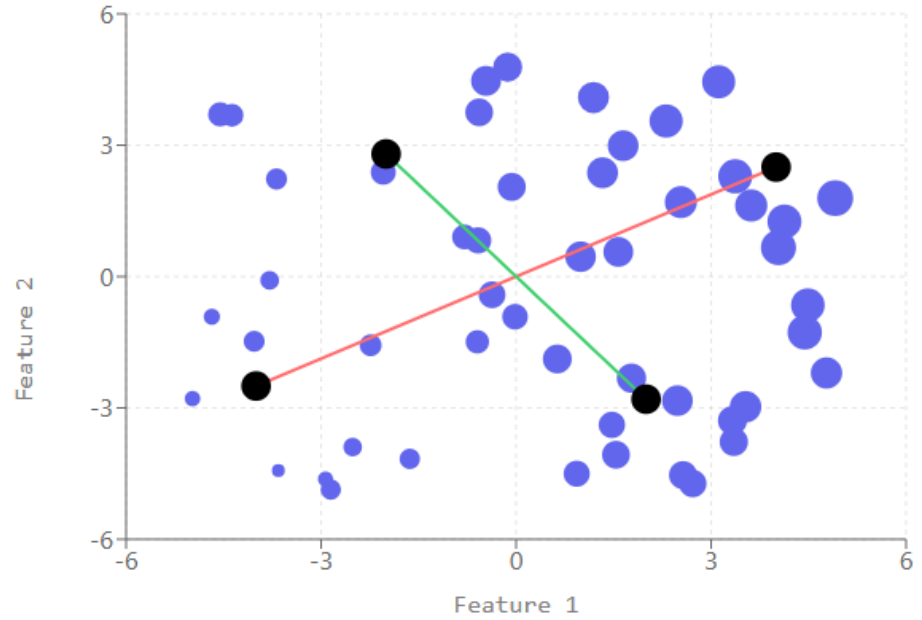
## Feature extraction

- Combine all features into a few new ones
- **PCA:** linear; finds the directions of greatest variance
- **UMAP:** non-linear; random, so fix random_state
- Unsupervised: uses no labels

# Unsupervised ML: Principal Component Analysis

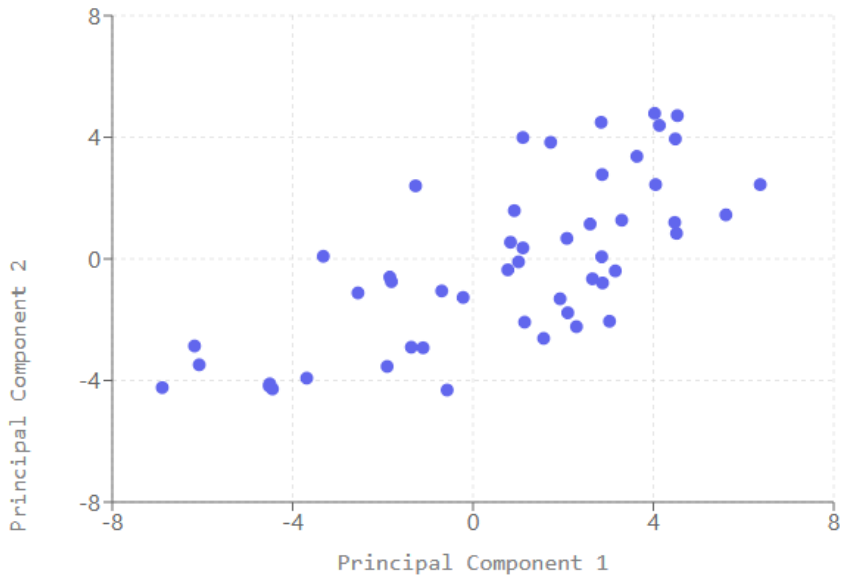
Transforms a set of correlated variables into a set of uncorrelated variables called principal components.

→ Finding Principal Components



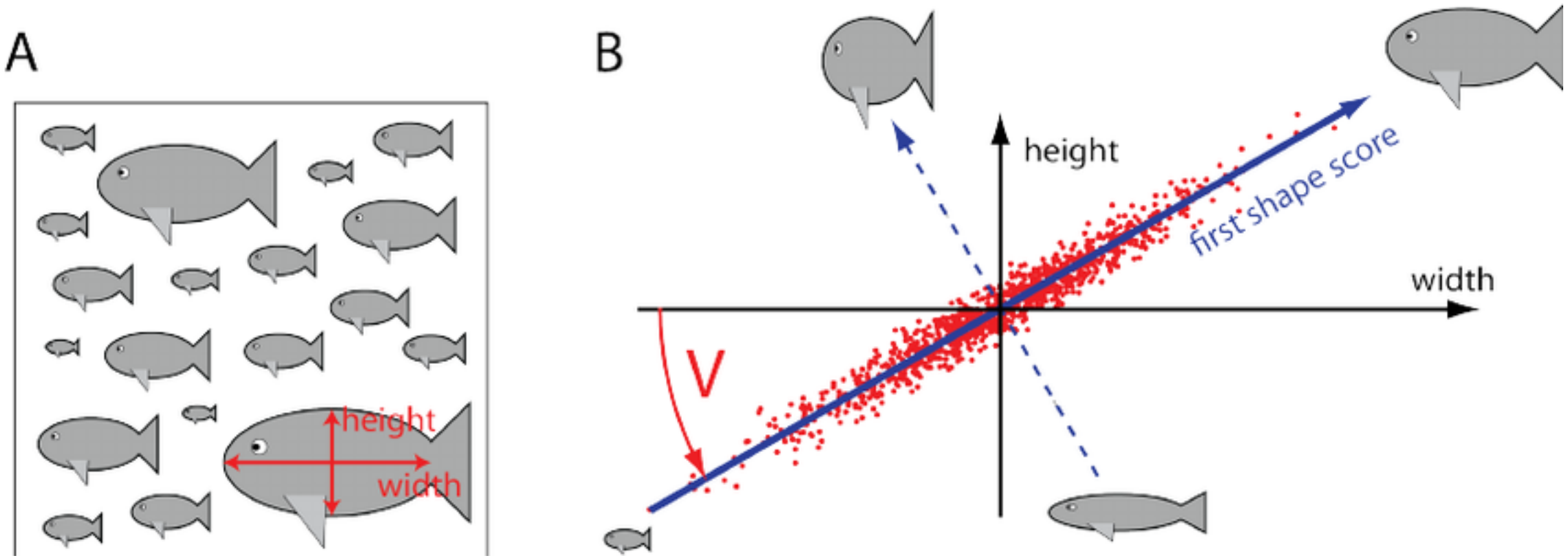

*PCA finds the directions of maximum variance in the data*

→ Projected 2D Data (After PCA)



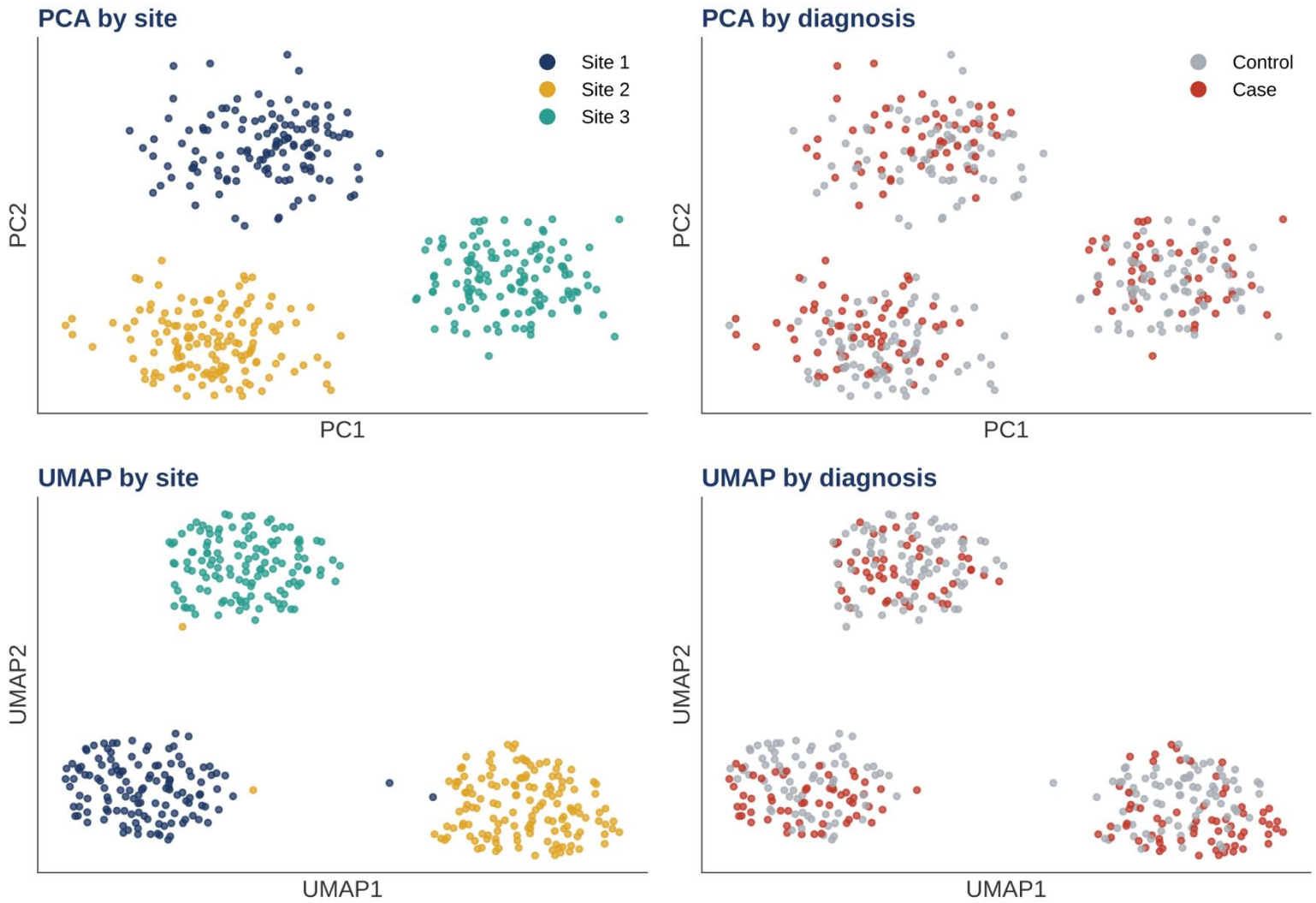

*Data is now represented in 2D while preserving most of the structure*

# Unsupervised ML: Principal Component Analysis



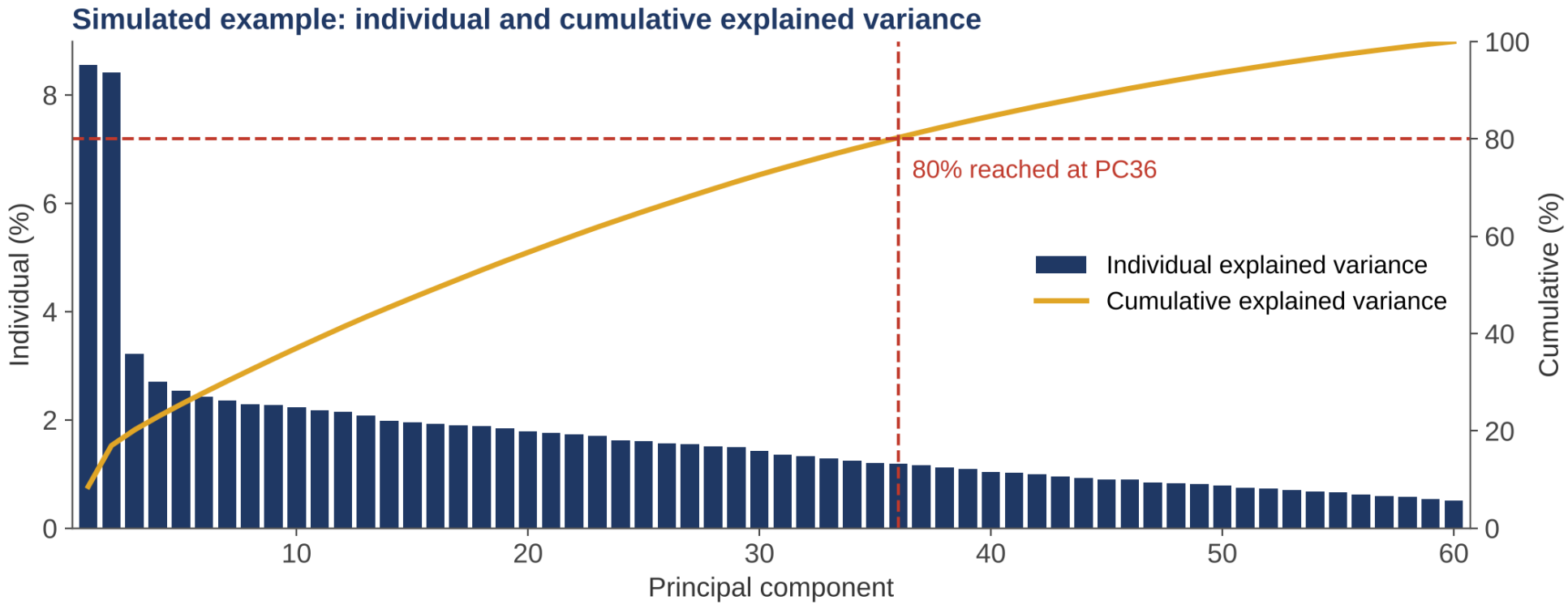
# PCA and UMAP: Many Features in Two Dimensions

*Simulated example: 400 people, 60 features*



# How Many Principal Components?

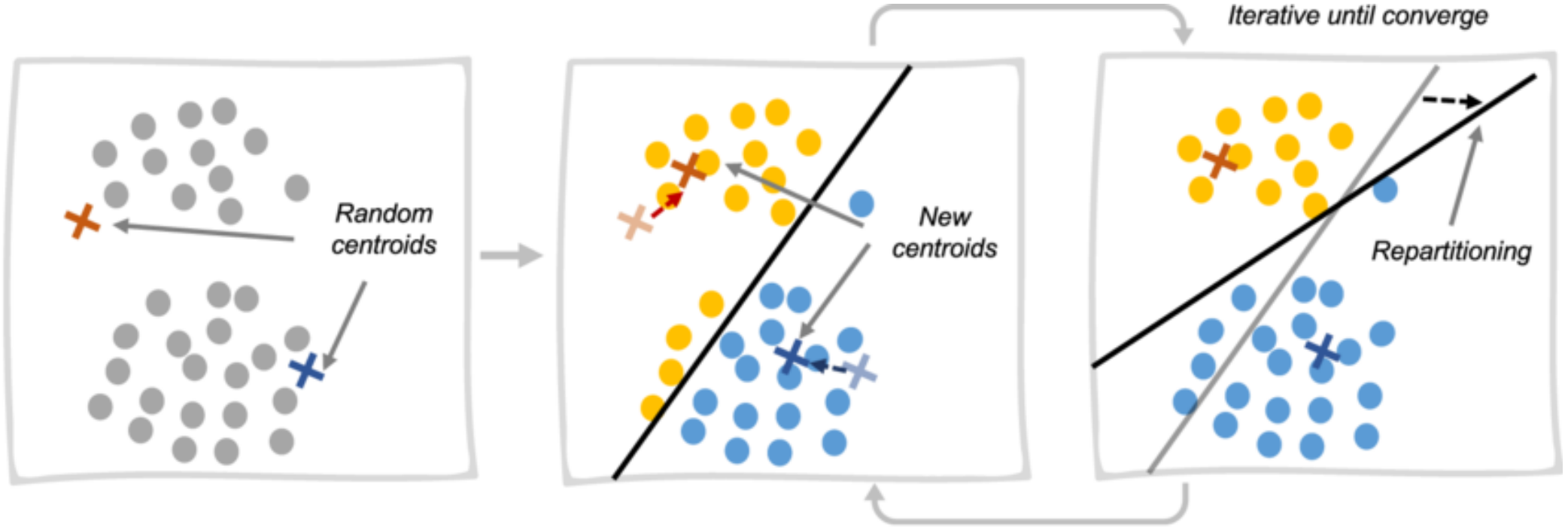
- **Individual explained variance:** how much each PC captures
- **Cumulative explained variance:** how much the first k PCs capture together




**Keep the PCs before a drop, or set a cut-off such as 80% of the variance.**

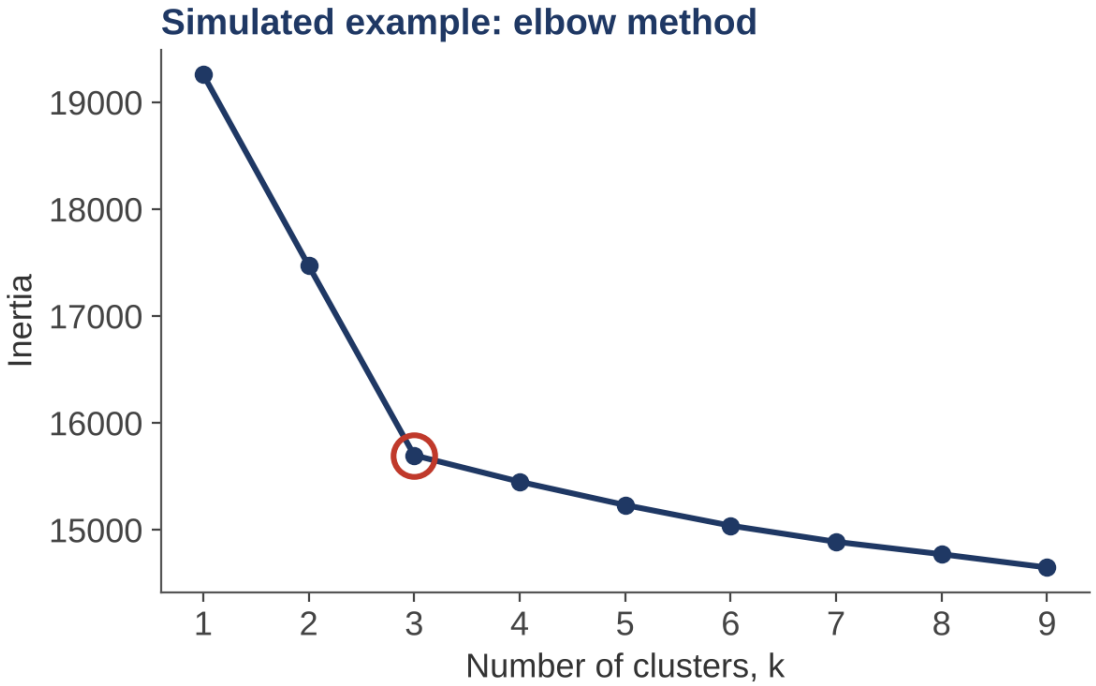
# Unsupervised ML: K-means Clustering

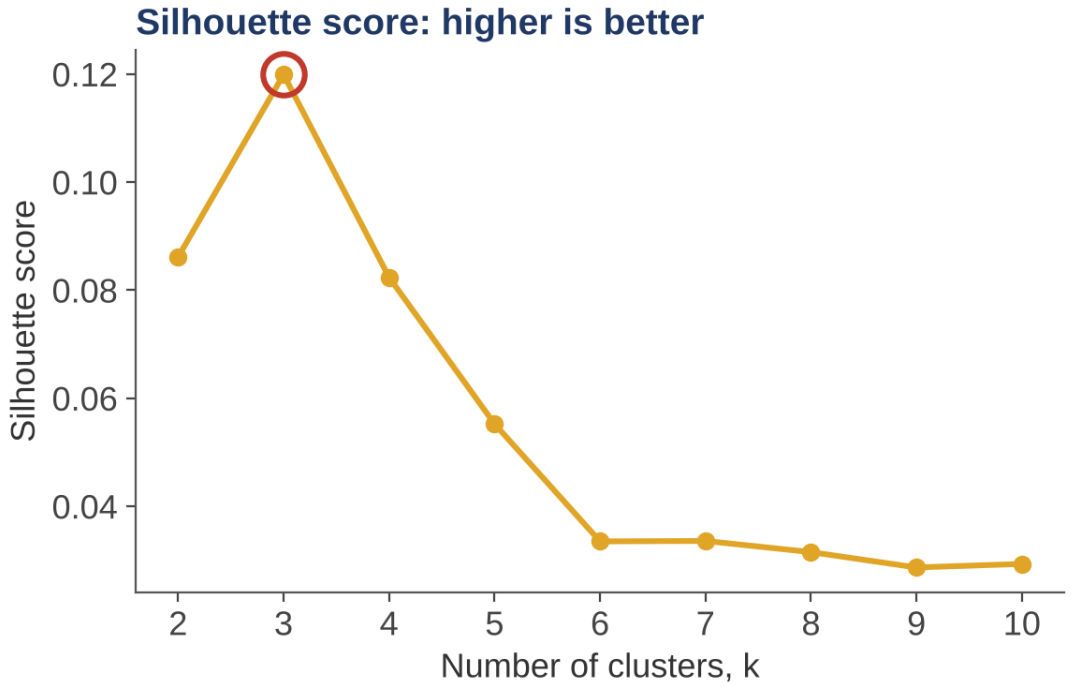
Application: identify disease subtypes




**Input: distance matrix *D* & number of clusters *k***

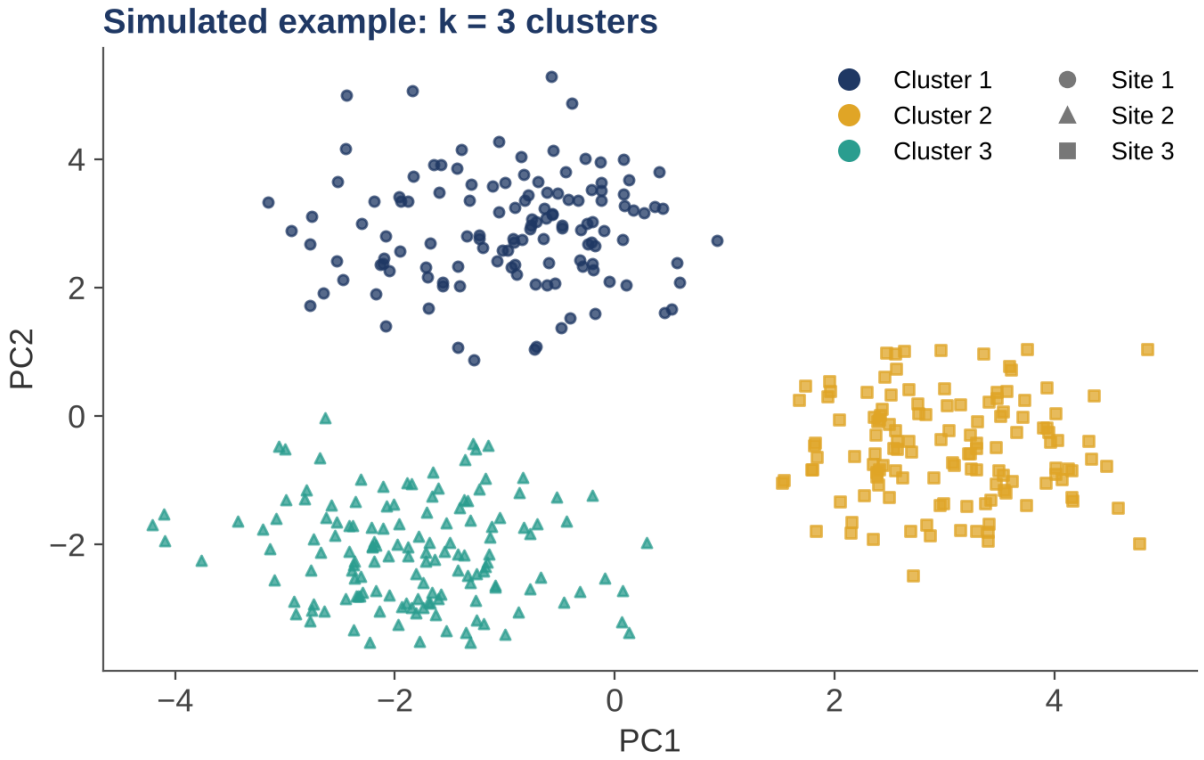
# How Many Clusters?

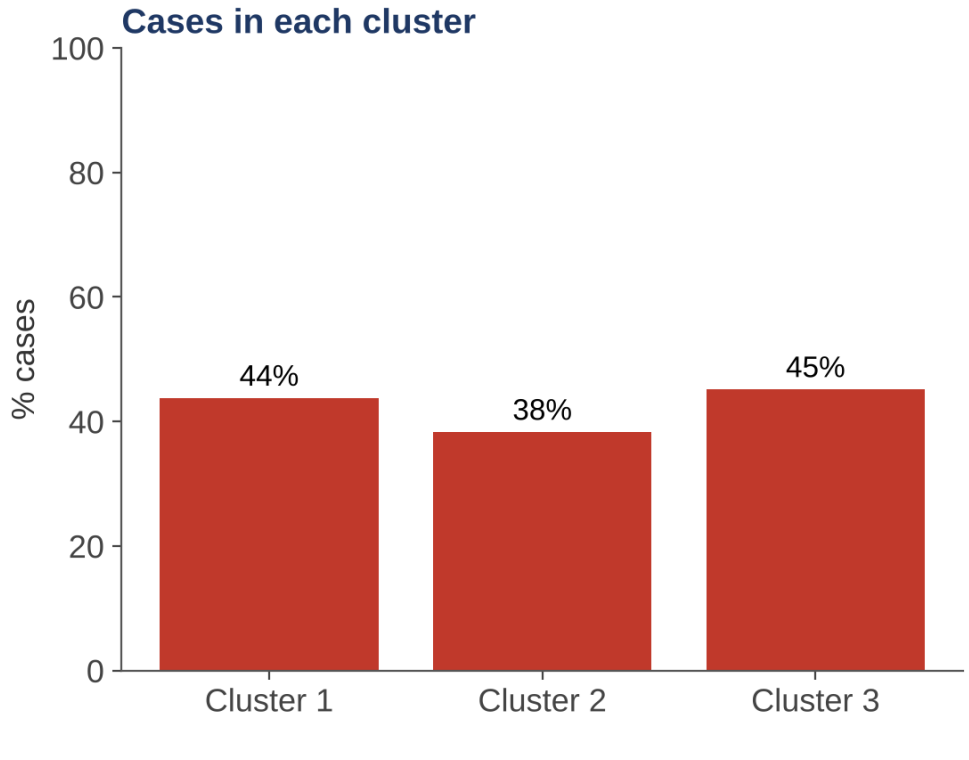



- **Inertia:** how tightly points sit around their centre; it always falls as k rises, so look for the elbow
- **Silhouette score:** how well separated the clusters are; higher is better
- Low silhouette scores mean the groups overlap: check the clusters against what you know

# Interpreting Clusters




- **Clustering finds the strongest structure first,** which may not be the one you are interested in.
- **Here the clusters are the three sites,** not cases and controls: each cluster has a similar share of cases.

# Take-home Messages

- **Supervised learning learns from labels:** regression predicts numbers (MAE, MSE, $R^2$); classification predicts classes (confusion matrix, sensitivity, specificity, precision, accuracy)
- **Avoid data leakage:** scale, impute and select features on the training set only
- **More features do not always help:** the curse of dimensionality
- **Unsupervised learning finds the strongest structure first:** check clusters against what you know, and choose k with the metrics and the clinical question

# Let's Code!

**Tutorial 4:** https://github.com/Sandoretal/Module3_2026/tree/main/tutorial_4

- **Open Intro_toML.ipynb:** Task 1 (regression), Task 2 (classification), Task 3 (PCA, UMAP and k-means)
- **Then the Challenge** at the end of the notebook: plasma proteomics and intestinal organ ageing
- **The data** are in the tutorial_4/data folder
- **Solutions** are released the next day: download the ZIP again to get them